Workshop: Mainframe Data Manipulation with Python

Welcome! In this workshop, you will build a Python service that connects to a mainframe, downloads a dataset, removes Social Security Numbers (SSN) entirely, and uploads the "clean" data back.

Why are we doing this?

Mainframes hold the world's most sensitive data. Traditionally, accessing this data required 3270 terminals. By using Python and the Zowe SDK, we can:

1. **Modernize Access**: Interact with the mainframe using standard REST APIs.
2. **Automate Compliance**: Programmatically remove information before it ever leaves the secured environment.
3. **Bridge the Gap**: Allow modern web applications to safely consume mainframe data in formats like JSON.

While we are reusing the Zowe APIs, we can also use them to create additional functionality. In this case, we are manipulating the data. This could be anything, such as capturing specific output, combining output, reformatting it to another useful format or even build large logic and integrations.

This workshop focuses on: Using Python to integrate with the Zowe APIs Learning how to publish APIs, useful if integrating with Zowe API Mediation Layer Creating a client to interact with the newly created APIs Managing credentials

---

Step 1: Install the Tools (5 minutes)

First, we need to install the Python libraries that do the heavy lifting. This command installs all the dependencies (additional libraries) so we can focus on the solution.

1. Open your terminal.

- Access it from the hamburger menu (three horizontal lines in the upper right)
- Select 'Terminal' then 'New Terminal'.

2. Navigate to this `scaffold/` folder.

```
cd scaffold
```

3. Run this command:

```
pip install -r requirements.txt
```

Step 2: Set Your Credentials (2 minutes)

We need to tell the code how to log in to your mainframe. You can use the file navigator to rename it and open it or use the terminal:

1. Find the file named `.env.example`.
2. Rename it to exactly `.env`. or `mv .env.example .env` at the terminal
3. Open `.env` and replace `your_username` and `your_password` with your real mainframe credentials.
4. Save the file.

Important Notes - Python Conventions

Python treats indention as logic blocks. Everything in the same block must be at the same indention level, with some exceptions, such as long strings and function parameters. If copying and pasting code, the indentation may not be correct. If you highlight the section of code, the tab key can move over the code block for you. And SPACEs and TABs are not always treated the same. For our purposes, use the TAB kep to be consistent.

Conventions:

Imports

```
from flask import Flask, request, jsonify
from zowe.zos_files_for_zowe_sdk import Datasets, DatasetOption
import logging
import os
```

- As described in the previous README.md, these libraries import functions into the applcation for execution
- Line 1 in manipulator_service.py imports Flask, our web framework.
- Line 1 in manipulator_service.py imports the Zowe framework for Datasets.

Functions

```
def name(parameter):
```

- This defines a function. All the function's code starts at the next line and is indented (1 TAB). You'll notice all the `def`s are in the first column and all the function's code is indented from there.
- See line 11 in manipulator_service.py.

Comments

Multi-line help is defined with three double-quotes(") at the start and end. Usually on the line by itself:

```
"""
HELP
"""
```

- The help describes what that function is performing.
- See line 12 in manipulator_service.py.

```
num = 4 #comment
```

- In-line comments start with a hash or pound sign (#) and everything after the # is treated as a comment
- See line 55 in in manipulator_service.py.

Returning Data

```
return data
```

- Indicates return data
- See line 24 in manipulator_service.py.

String Work []

```
string[0:8]
```

- These are string operations, short hand to capture part of the string, in this case the first 9 characters.
- See line 59 in in manipulator_service.py.

Lists []

```
names = [item.name for item in items]
```

- [and] are used as lists, when not assigned to a string.
- This code snippet looks through a list called items, generates a list of names (item.name), and return it to a list to a variable called names.
- This is a very Pythonic function and is shorthand for doing a long for loop

Try/Exception Blocks

```
try:
    something
except Exception as e:
    something
```

- This is exception handling. The code block in `try` executes. If there's a failure, the `exceoption` block is called. Otherwise it is skipped.
- Exception handling eases logic for failures and is good programming.
- See line 77 in in manipulator_service.py.

Decorators

```
@app.route('/list-datasets', methods=['POST'])
```

- Decorators are a concise way to wrap or modify the behavior of a function (or method) without changing its source code.
- This decorator was defined in the Flask module. It defines the web endpoint (http://localhost/list-datasets) and the method(s) allowed, in this case "POST"
- The next line is almost always a function (`def function():`). When this endpoint is accessed, it calls the function immediately after it.
- See line 89 in in manipulator_service.py.

Addtional context

More context will be added in the document for things that aren't completely clear.

Phase 1: Server Implementation (`manipulator_service.py`)

In this phase, we will build the "brain" of our service. We'll implement the manipulation logic, the dataset allocation helper, and the API endpoints.

Step 3: Task 1 - Implement Manipulation Logic

Goal: Create the "brain" that knows how to strip SSNs from fixed-width COBOL data.

- This is a helper function. It accepts a parameter (content) and returns a modified version of the data.
- In the US, you may know, there's a Social Security Number (SSN). We want to remove it from our dataset.
- This function takes a large string, removes the SSN, and returns the data.
- It does not interact with the dataset directly. It assumes any content passed is a string of the given format.

1. Open `manipulator_service.py`.
2. Find the `manipulate_data` function (around line around 32).
3. Replace the `TODO: Task 1` sections inside the `for` loop with this code:

```
    # Remove SSN (chars 12-22) by skipping them
    rest = line[23:93]

    # Reconstruct the fixed-width line without SSN
    new_line = f"{account}{bill}{rest}"
```

Why? Mainframe data is often "fixed-width," meaning every field starts and ends at an exact character position. By slicing the string to skip the SSN positions, we create a new, safe record format.

Step 4: Task 2 - Implement Dataset Allocation

Goal: Ensure the mainframe has a place to store our manipulated data.

- This uses Zowe's Dataset library
- We get a list of datasets and returns into a response record.
- `if not any(...)` searches for the dataset name, if it is not found, it creates the dataset using `datasets.create(...)`.
- Using Zowe, dataset parameters can be set as desired, just like on the mainframe. While using sequential datasets here, it could be a PDS or any other dataset type.

NOTE: If you copy and paste this, it will not be formatted correctly (it won't be indented). You will see a number of squiggly lines in the code. Highlight the section, then press tab until the squiggly messages go away, about 2 times. Or, even better, type it in.

1. Find the `ensure_dataset_exists` function (around line 72).
2. Implement the logic to check and create:

```
response = datasets.list(dsname)
if not any(item.dsname.upper() == dsname.upper() for item in
response.items):
    options = DatasetOption(
        dsorg="PS", recfm="FB", lrecl=lrecl,
        blksize=7161, primary=1, secondary=1, alcunit="TRK"
    )
    datasets.create(dsname, options=options)
```

Why? Unlike modern file systems, mainframe datasets must be "allocated" with specific attributes (like LRECL - Logical Record Length) before they can be written to.

Step 5: Task 3 - Implement List Datasets Route

Goal: Create an API endpoint to verify our connection.

- Here, we are implementing our first API endpoint. The `@app.route` line defines the enpoint.
- The function, `handle_list_datasets` expects a JSON request with `hlq` defined.
- `data = request.json` retrieves the json data from the header.
- `hlq = data.get('hlq')` tries to retrieve the value. In the lines below, it makes some tests. If the tests fail, it returns a 400 error.
- We are implementing a simple routine to get a list of datasets

1. Find the `handle_list_datasets` function (around line 85).
2. Implement the logic to list datasets:

```
datasets = Datasets(profile)
response = datasets.list(f"{hlq}.*")
names = [item.dsname for item in response.items]
return jsonify({"status": "success", "datasets": names})
```

Why? This teaches us how to wrap a Zowe SDK call into a Flask web route, making mainframe data accessible via a simple HTTP request.

Step 6: Task 4 - Implement Main Manipulation Route

Goal: The core workflow—Download -> Manipulate -> Upload.

- This is the core workflow the service
- This function downloads the dataset to the server, runs the manipulate data function, then uploads the dataset back to the mainframe.
- Rather than making the user validate the dataset exists, we will create it for them if it doesn't.
- Because we have the helper functions, this is pretty simple. Get the dataset name from the header, check for values, then run the proceess.
- The `try:` block checks to ensure the dataset exists and creates it if it doeesn't. It writes the input dataset to disk, reads it, then uploads the modified content to the new dataset.
- Then returns a proper return code if successful, otherwise it sends an error.
- `open(...)` is a function for opening files for reading or writing, depending on the flags.
- And `finally:` is part of the try/exception block. Once the code has run (both success and failure), it cleans up the disk.
- *Note: This service is **not thread-safe**, meaning it should only do one request at a time.*

1. Find the `handle_manipulate` function (around line 105).
2. Implement the logic to download, manipulate, and upload:

```
datasets = Datasets(profile)
# Ensure output dataset exists with LRECL 82 (since SSN is removed)
ensure_dataset_exists(datasets, output_ds, lrecl=93)

datasets.perform_download(input_ds, temp_in)
with open(temp_in, "r") as f:
    content = f.read()

manipulated_content = manipulate_data(content)

with open(temp_out, "w") as f:
    f.write(manipulated_content)
datasets.perform_upload(temp_out, output_ds)
```

Why? This is the automation "magic." We pull sensitive data into memory, clean it using our "brain" from Step 3, and push the safe version back to a new dataset.

Step 7: Task 5 - Implement Download Dataset Route

Goal: Expose the cleaned data to the outside world.

- This function reads the target dataset, saves it to disk, then posts it to the response.
- Much like the previous calls, it gets the header information and validates it.
- `datasets.perform_download(...)` downloads the dataset to disk using Zowe's API.
- It reads the file and sends it the the payload for the client side.

1. Find the `handle_download_dataset` function (around line 143).
2. Implement the logic to download and return raw content:

```
datasets = Datasets(profile)
datasets.perform_download(input_ds, temp_in)
with open(temp_in, "r") as f:
    content = f.read()
return jsonify({"status": "success", "content": content})
```

Why? This allows a client application to fetch the "clean" data directly, which we will then parse into JSON.

The server side has been implemented.

Phase 2: Client Implementation (`client.py`)

Now that our server is ready, we'll build the client to interact with it.

Step 8: Task 6 - Implement Credential Helper

Goal: Securely handle mainframe authentication.

- This client application can prompt for infomration, use an environment file, or even connect to the Zowe configuration files.
- To keep things simple, we are using environment variables or prompting.
- Just like the server side, we are importing libraries. Since this is the client side, we don't need to import the Zowe libraries.
- `load_dontenv()` uses the `.env` file to read configuration information.
- `payload = {...}` creates a dictionary variable. Dictionaries have a named field and a value. The first reference creates blank values, the second reference assigns the variables to those dictionary fields.
- Between the `payload = {...}` calls, the variables are assigned either through the `.env` file or through prompting the user.

1. Open `client.py`.
2. Find the `create_payload` function (around line 10).
3. Replace the code inside with:

```
username = os.getenv("MAINFRAME_USER") or input("Enter Username: ")
password = os.getenv("MAINFRAME_PASSWORD") or getpass.getpass("Enter
Password: ")
```

```
host = os.getenv("MAINFRAME_HOST", "mainframe.example.com")
port = int(os.getenv("MAINFRAME_PORT", 5010))
reject_unauthorized = os.getenv("MAINFRAME_REJECT_UNAUTHORIZED",
"false").lower() == "true"

payload = {
    "username": username, "password": password, "host": host,
    "port": port, "reject_unauthorized": reject_unauthorized
}
if extra_data:
    payload.update(extra_data)
return payload
```

Why? We want to support both automated environments (using `.env` files) and interactive use (prompting for passwords) without hardcoding sensitive info.

Step 9: Task 7 - Implement Main Execution Logic

Goal: Use our client to trigger the server-side tasks.

- Defines the execution logic.
- Gets a list of datasets using the HLQ
- Calls the manipulate_data function
- Downloads the manipulated data and writes it to a file.

1. Find the `main` function (around line 36).

2. Replace the `TODO: Task 7` sections with this code:

   Task 1: List Datasets

   ```
   hlq = os.getenv("MAINFRAME_USER") or input("Enter HLQ to list: ")
   list_payload = create_payload({"hlq": hlq})
   list_response = requests.post("http://localhost:5010/list-datasets",
   json=list_payload)
   print("Found datasets:", list_response.json().get("datasets", []))
   ```

   - The data is just being printed on screen

   Task 2: Manipulate Data

   ```
   manipulate_payload = create_payload({
       "input_dataset": input_ds,
       "output_dataset": output_ds
   })
   response = requests.post("http://localhost:5010/manipulate",
   json=manipulate_payload)
   print(response.json().get("message", "Manipulation complete!"))
   ```

- The

Task 3: Download and Display JSON

```
download_payload = create_payload({"input_dataset": output_ds})
download_response = requests.post("http://localhost:5010/download-
dataset", json=download_payload)
content = download_response.json().get("content", "")

records = []
for line in content.splitlines():
    if len(line) >= 93:
        records.append({
            "account": line[0:8].strip(), "bill": line[8:12].strip(),
            "name": line[12:42].strip(), "phone": line[42:52].strip(),
            "email": line[52:82].strip()
        })
print(json.dumps(records, indent=2))
```

Why? Task 3 is the ultimate payoff. We take the raw, fixed-width mainframe data and transform it into a modern JSON format that any web developer can use.

Step 10: Run the Workshop! (8 minutes)

Now we put it all together.

1. **Start the Service**: In your terminal, run:

   ```
   python manipulator_service.py
   ```

   You should see "Running on http://0.0.0.0:5010". **Leave this terminal running.**

2. **Run the Client**: Open a **second terminal** window, navigate back to this `scaffold/` folder, and run:

   ```
   python client.py
   ```

   If you didn't set up the `.env` file, it will ask for your username and password.

3. **Verify the Result**: The client should say "Success!". Now, log in to your mainframe and check the `ZOWEUSER.PUBLIC.MANIPULATED.DATA` dataset. The SSNs should be gone, and each record should now be 93 characters long instead of 93!